Milestone 1

Team Member	Component(s)	Specific Tasks Completed	Testing/Debugging	GitHub Contributions
Tobias Harold	Underlying Data structures (linked list, queue, stack), general debugging, and github merge management.	Developed the linkedlist template, built the queue and stack on top of it, added the baseline structures for the reservation class, and tested for logic errors in all files.	Parsed over the final implementation and connection of the main file, made sure all edge cases were able to be tested, and developed the make file for cross platform compatibility.	Merge conflict resolution, hosting of the repo, makefile, .gitignore, and .gitattributes implementation, and baseline development of the repo.
Adrian Ochoa	Reservation functionality and complexity analysis.	Created the ReservationManager class and implemented reservation creation, validation, cancellation, and displaying active reservations. Added duplicate reservation ID checks and resource/date conflict validation. Added reservation comparison support for linked-list removal. Wrote the complexity analysis for reservation insertion/removal, waiting-list processing, and undo cancellation.	Tested reservations, rejecting dup/conflicting reservations, displayed active reservations, cancelling reservations and confirming the remaining res-list updated correctly	created and worked on the reservation-functionality branch. Added and updated. Test code in main.cpp and the complexity analysis file.

Roshan Sharma	Resource management and user menu	Implemented the Resource and ResourceManager functionality to retrieve data about the resource from the file and display it on screen. Developed the menu prompting and selection logic in main.cpp and connected menu choices to the group's existing function. Added handling for invalid menu input	Tested loading and displaying the provided resource data. Tested each menu selection and verified that the menu correctly called the existing reservation, waiting-list, cancellation, and undo functions.Debugging invalid text input that caused the menu to repeat continuously and fixed a CMake build issue involving Cancellations.cpp	Developed resource management on the resource-management branch and menu integration on the menu-integration branch. Created commits and pull requests for both components. Updated CMakeLists.txt to include Cancellation.cpp

Tobias Harold

I worked mostly on the initial setup and implementation of the underlying structures. Initially, I created the .gitignore and .gitattributes in order to allow clean usability across windows and linux. Additionally, I developed the standing makefile template to use when compiling, to allow for ease of error checking across different platforms. Then, I shifted to developing the data structures that everything else would be built on to get it finished early. I built out a linked list class for Reservations specifically, before realizing I would need to do the same 3 more times for all other list structures. So, I switched to a linked list template file instead, and built queue, stack and resources list on top of it too. I then took a step back and mostly debugged while the others wrote their classes, pulling all of it together and resolving merge conflicts when they arose. At the end of the project, I troubleshot logic errors and tested it on the cell machines to verify functionality, and put build steps in the README for ease of testing.

I worked on the reservation functionality and complexity analysis. Created the resManager class and implemented functions for creating, validating, cancelling and displaying reservations. I added checks to provent dups. Added comparison support so reservations could be removed from the linked list correctly. Tested the res system by creating valid and invalid reservations. I also wrote the complexity analysis for reservation insertion, removal, waiting list processing, and undo cancellation. My work was done on the reservation-functionality branch.

Roshan Sharma
I worked mainly on resource management and the user menu. I implemented the Resource and ResourceManager functionality, store the resource information, and display the resources to the user. I later developed the menu prompting and selection logic in main.cpp. For the menu integration, I connected the menu choices to functions that had already been implemented by my teammates. I tested the menu options and resource display and debugged an invalid-input problem where entering text instead of a number caused the menu to repeat continuously. Also fixed a build issue by including Cancellation.cpp in the CMake configuration.

Final Submission

Team Member	Component(s)	Specific Tasks Completed	Testing/Debugging	GitHub Contributions
Tobias Harold	Reporting functionality and menu fixes; integration of searching and sorting	I build the report generator files, and build the report sub menu in the menu to display the required reporting statistics after the resources had been sorted or corrected.	I tested most of the menu variable type edge cases, including miscellaneous inputs in the menu so that it is robust and can handle every type of input without crashing the program.	Contributed mostly to merging pull requests and resolving code conflict, making sure that all methods are accessed and pulled into a menu that the user can use to present the functionalities.

Adrian Ochoa	Sorting Function, and resource management integration	Built the merge sort for resource data and refactored it into a reusable template. Added sorting by alphabetically by name and implemented case insensitive name comparison.	Compiled and tested the sorting implementation throughout development, fixed template syntax. Verified that resources were sorted alphabetically and tested interaction with the resource ID search	Made the sorting feature on the sorting functionality branch and did many testing and debugging
Roshan Sharma	Binary Search implementation, resource searching, waiting list withdrawal menu, and complexity analysis	Implemented Binary Search to find resources by their IDs. Integrated the search functionality into the main. Added an option to withdraw students from the waiting list using the existing dequeue function. Added binary search time and space complexity analysis.	Tested the Binary with valid and invalid resources IDs. Checked if searching works even after displaying resources sorted by name. Tested waiting list withdrawal, including FIFO behavior and empty queue handling.	Developed the searching-algorithm branch, committed and pushed the Binary Search and waiting list menu updates, updated complexityAnalysis.

Tobias Harold

My tasks for the second phase of the project mostly revolved around reporting functionality and the menu usage with other methods. The reporting functionality was built mostly around the display methods in other management classes, simply making sure that it is sorted/searched

properly before presenting the data. Additionally, the waiting list needed rework as there was only a single one, and there needed to be separate queues based on resource and date. It was decided that a nested map could be used for quick parsing and accessing of specific queues that the user wanted to view. Additionally, the submenu for reporting was developed by me, simply calling the correct methods as reporting is mostly a static class, with almost no need for multiple objects of that class. The menu also was missing a few options/ had some unnecessary options, so the menu was cleaned and reorganized to suit the new functionality.

Adrian Ochoa

Mainly responsible for the sorting functionality of the p project, implemented merge sort for the resource list and then made a generic template that used a comparator. That made the same sorting algorithm to work with different data types or sorting rules. The comparator sorted the resources alphabetically by name and later made it case insensitive. During testing I found that changing the order of the main resource vector could interfere with the binary search by resource ID and other logic so I adjusted the sorting design so that the sorted data could be used without modifying the main resource. I completed this work on the sorting functionality branch and made multiple commits and pushes throughout.

Roshan Sharma
I mainly focused on developing Binary Search for resources and enhancing the user menu . Implemented Binary Search to find resources using their IDs and connected it to the main. I also added the waiting list withdrawal option using the existing dequeue function for fixing the bug identified in milestone 1. I updated the complexity analysis report with the information about time and space complexity of Binary Search. I also tested the searching functionality with valid and invalid resource IDs and ensured that the same works fine after sorting resources by name. I have tested the waiting list withdrawal functionality to ensure that students are deleted in the correct order and an empty waiting list has been handled properly. Completed everything on the search-algorithm branch and created a pull request to merge my changes into the main project.